

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

**Duration :60 Mins
On Class
Total Marks:50**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I Vishal. B(192524485) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Vishal.B

CO1 - Analytical Problems.

1. $W = x * y - z + 20$

Solu:

$W = x * y - z + 20$

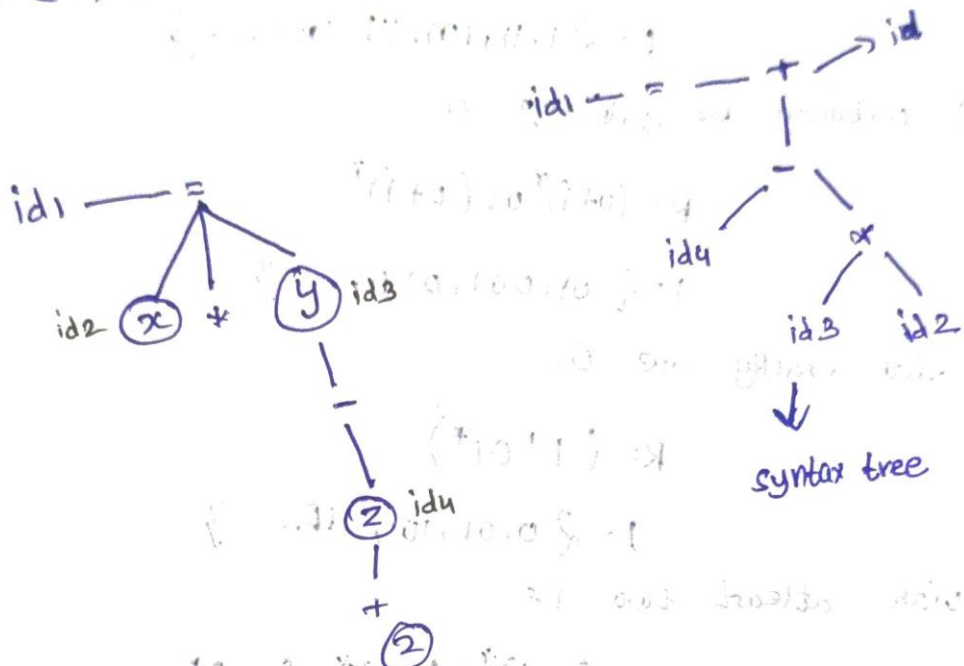
Syntax : $S \rightarrow id = E$

Analyzer : $E \rightarrow E + E \mid E * E \mid E - E \mid E / E \mid id$

Lexical : $\langle id_1 \rangle \langle = \rangle \langle id_2 \rangle \langle * \rangle \langle id_3 \rangle \langle - \rangle \langle id_4 \rangle \langle + \rangle \langle 20 \rangle$

Analyzer

parse tree :



a. Intermediate code Generator :

$t_1 = id_2 * id_3$

$t_2 = t_1 - id_4$

$t_3 = t_2 + 20$

$id_1 = t_3$

↓

code optimizer Generator

$t_1 = id_2 * id_3$

$id_1 = (t_1 - id_4) + 20$

↓

code generator

mov id2, R1

mov id3, R2

mul R2, R1

SUB id_4, R_1

ADD $\#20, R_1$

MOV R_1, id_1

Q. Q2:

i.) All the strings beginning and ending with 1

$$R = 1(0+1)^*1$$

$$L = \{1, 11, 101, 111, 1001, \dots\}$$

ii.) containing the substring 01

$$R = (0+1)^*01(0+1)^*$$

$$L = \{01, 001, 0101, \dots\}$$

iii.) with exactly one 0.

$$R = (1^*01^*)$$

$$L = \{0, 01, 10, 110, \dots\}$$

iv.) with atleast two 1's

$$R = (0+1)^*1(0+1)^*1(0+1)^*$$

$$L = \{11, 101, 110, 1110, \dots\}$$

v.) All strings that don't end with '0'

$$R = \epsilon + (0+1)^*1$$

3. Languages Generated:

i.) $(\epsilon + aa^*)^*$

$$L = \{a^*\}$$

eg: $\epsilon, a, aaaa, \dots$

ii.) $(a^*b^*)^*abb$

$$L = \{(a+b)^*abb\}$$

eg: $abb, aabb, bbabb, \dots$

iii.) $(a^*b^*)^*aba$

$$L = \{(a+b)^*aba\}$$

eg: $aba, aaba, bbaba, \dots$

$\therefore \boxed{L = aba} \rightarrow$ any combination of aba

iv) $(a+ab)^*$

$L = \{ \epsilon, a, ab, aa, aab, \dots \}$

v) $(abc)^* + (a^*b^*)^*$

$L = \{ \epsilon, a, b, aaa bbb, abb, \dots \}$

4.

i) Lex regular Expression for identifier.

$[A-Za-z][A-Za-z0-9]^*$

* $[A-Za-z] \rightarrow$ The first character must be a letter.

* $[A-Za-z0-9]^* \rightarrow$ The first character must be a letter remaining characters can be letters or digits.

ii) Keywords :

* If

* Else

* While

} Keywords

"*" $\rightarrow$ operator

"+" $\rightarrow$ operator

"{" $\rightarrow$ delimiter

"[]" $\rightarrow$ delimiter

* Program :

% 2

#include <stdio.h>

% }

letter [A-Za-z]

digit [0-9]

identifier { letter } ({ letter } { digit }) *

% %

If { printf("Y text"); }

while { printf("Y text"); }

else { printf("Y text"); }

{ id } { printf("Y text"); }

[0-9] { printf("Y text"); }

```
"+" | "-" | "/" | "*" } printf("%s", text); }
```

```
" (" | ")" | "{" | "}" | "[" | "]" | "<" | ">" } printf("%s", text); }
```

```
{ if (n) { } printf("%s", text); }
```

```
%s %s
```

```
int main () {
```

```
printf ("Enter");
```

```
yytext ();
```

```
return 0; } int yywrap () {
```

```
return 1;
```

5. i.) $(0^*1^*)^* = (0+1)^*$

$$(0^*1^*)^* = \{ \epsilon, 0, 1, 00, 01, 10, \dots \}$$

$$(0+1)^* = \{ \epsilon, 0, 1, 00, 01, 10, \dots \}$$

$$(0^*1^*)^* = (0+1)^*$$

$$LHS = RHS.$$

ii.) $(0+1)^* (0+1)^* = (0+1)^*$

$$R^* R^* = R^*$$

$$\text{So, } (0+1)^* (0+1)^* = (0+1)^*$$

$$LHS = RHS.$$

iii.) $(0^*)^* = 0^*$

$$(R^*)^* = R^*$$

$$\text{So, } (0^*)^* = 0^*$$

$$LHS = RHS.$$

iv.) $(0+1)^* 0 (0+1)^* + (0+1)^* 1 (0+1)^* = (0+1)^*$

$$(0+1)^* (1+0) = (0+1)^*$$

$$(0+1)^* = (0+1)^*$$

$$LHS = RHS.$$

v.) $\epsilon + 0(0)^* = 0^*$

$$\epsilon + R(R^*) = R^*$$

$$\text{So, } \epsilon + 0(0)^* = 0^*$$

$$LHS = RHS.$$